Tuning BM25: k_1 & b, Worked Out by Hand

The two knobs that shape every BM25 score — k_1 controls how fast repeated terms stop helping, b controls how hard long documents are penalised — isolated, tabulated, and read off real numbers.

What this document does. It takes the BM25 formula (#15) and varies one parameter at a time. A term-frequency saturation table shows what k_1 does; a length-normalisation table shows what b does, including the special points $b = 0$ and $b = 1$. It closes with practical tuning guidance. Numbers from Appendix A.

Prerequisite: the BM25 formula (#15).

1 Where the two knobs live

Recall a single term's BM25 contribution:

$$\text{idf}(t) \cdot \frac{\text{tf}\,(k_1+1)}{\text{tf} + k_1\underbrace{\left(1 - b + b\,\frac{|d|}{\text{avgdl}}\right)}_{\text{length factor}}}.$$

k_1 appears in the saturation of tf; b appears only inside the length factor. They are independent dials on two different behaviours.

> **Intuition**
>
> k_1 answers "*how much does the 5th mention of a word add over the 1st?*" Low $k_1 \to$ almost nothing (one mention is enough, the rest saturate fast); high $k_1 \to$ repetition keeps mattering. b answers "*how much should I distrust a long document?*" $b = 0 \to$ length is ignored entirely; $b = 1 \to$ length is fully divided out. Defaults $k_1 = 1.5$, $b = 0.75$ are a well-tested compromise, not a law.

2 k_1 — the saturation knob

Isolate it: set idf $= 1$ and $b = 0$ (no length effect), so the contribution is $\frac{\text{tf}(k_1+1)}{\text{tf}+k_1}$. Sweep term frequency:

tf	1	2	3	5	10	20
$k_1 = 0.5$	1.000	1.200	1.286	1.364	1.429	1.463
$k_1 = 1.2$	1.000	1.375	1.571	1.774	1.964	2.075
$k_1 = 2.0$	1.000	1.500	1.800	2.143	2.500	2.727

Read across any row: the jump from tf $= 1$ to 2 is large, but $10 \to 20$ barely moves — that is saturation. Read down any column: bigger k_1 lets repetition keep paying off (at tf $= 20$, k_1=2 gives 2.73 vs k_1=0.5's 1.46). Each curve approaches its ceiling $k_1 + 1$ (here 1.5, 2.2, 3.0) but never reaches it.

Mini-example · $k_1 \to 0$ is binary matching

As $k_1 \to 0$ the factor $\frac{\text{tf}(k_1+1)}{\text{tf}+k_1} \to 1$ for *every* tf ≥ 1: the term either appears (score idf) or it does not (score 0). Counts stop mattering entirely — BM25 degenerates to "does the word occur?" Low k_1 suits fields where one mention is as good as ten (titles, tags); high k_1 suits long prose where repetition signals focus.

3 b — the length-normalisation knob

Isolate it: idf $= 1$, $k_1 = 1.5$, tf $= 3$, avgdl $= 12$. Vary b and the document length $|d|$:

	$\lvert d\rvert = 6$ (half avg)	$\lvert d\rvert = 12$ (avg)	$\lvert d\rvert = 24$ (2×)	$\lvert d\rvert = 48$ (4×)
$b = 0.00$	1.667	1.667	1.667	1.667
$b = 0.50$	1.818	1.667	1.429	1.111
$b = 0.75$	1.905	1.667	1.333	0.952
$b = 1.00$	2.000	1.667	1.250	0.833

Three things to see. (1) At $|d| = \text{avgdl} = 12$ *every* row is 1.667 — the length factor is exactly 1 at average length, no matter what b is. (2) The $b = 0$ row is flat: length is ignored, a 48-token document scores like a 6-token one. (3) As b rises, short documents get rewarded (column $|d| = 6$ climbs to 2.0) and long ones punished (column $|d| = 48$ falls to 0.83) — the spread widens around the average.

Intuition

The pivot at $|d| = \text{avgdl}$ is the key insight: b does not move average-length documents at all; it only stretches the reward/penalty for documents that deviate from average. So b is really "how suspicious am I of unusual lengths?" Corpora of uniform length (tweets, log lines) want low b; corpora mixing snippets and books want high b so a long book cannot dominate by sheer word count.

4 Practical tuning

- **Start at defaults** $k_1 = 1.5,\ b = 0.75$ — robust across most English text.
- **Raise** k_1 (toward 2–3) for long documents where repeated terms genuinely signal relevance; **lower it** (toward 0.5) for short fields where one hit is enough.
- **Raise** b (toward 1) when document lengths vary wildly and you see long documents winning unfairly; **lower it** (toward 0) for uniform-length corpora.
- **Tune empirically** against a labelled query set using nDCG / MRR (Track 3.1, 6.1) — never by eyeballing a few queries. The interaction with your specific length distribution matters more than any textbook value.

Equation cheat-sheet

$$\text{tf factor} = \frac{\text{tf}(k_1+1)}{\text{tf}+k_1} \nearrow k_1{+}1 \text{ (ceiling)}, \qquad k_1 \to 0 \Rightarrow \text{binary match}$$

$$\text{length factor} = 1 - b + b\frac{|d|}{\text{avgdl}}; \quad = 1 \text{ at } |d|{=}\text{avgdl}; \quad b{=}0 \text{ ignores length, } b{=}1 \text{ full}$$

Appendix A · Reference implementation

```
def tf_factor(tf, k1):                  # idf=1, b=0
    return tf * (k1 + 1) / (tf + k1)
[round(tf_factor(tf, 2.0), 3) for tf in [1,2,3,5,10,20]]
# [1.0, 1.5, 1.8, 2.143, 2.5, 2.727]   -> saturates toward k1+1 = 3

def len_factor_contrib(dl, b, tf=3, k1=1.5, avgdl=12):
    K = k1 * (1 - b + b * dl / avgdl)
    return tf * (k1 + 1) / (tf + K)
[round(len_factor_contrib(dl, 0.75), 3) for dl in [6,12,24,48]]
# [1.905, 1.667, 1.333, 0.952]         -> pivot of 1.667 at dl = avgdl = 12
```

Internal learning material. Numbers reproducible from the appendix code. v1.0